Trees, Search, and Decision Structures

A clean hierarchy can still hide a required relationship

COMSC 2043 ▪ Discrete Structures and Critical Thinking

Week 13, Tuesday, November 10, 2026

Week 12 continuity: the model comes first

PRINCIPLE: MODEL VS PROCEDURE

Carry this sentence forward

A correct procedure on the wrong model is still wrong.

- Last week, a one-way street meant the two directions had different edges.
- This week, a tree will make a tidy picture of relationships.
- Before tracing the picture, ask what one edge means.

PREDICT (30 s): If the arrows mean dependency but the tree edges mean containment, can the same picture answer both questions?

Today: model, trace, audit

PRINCIPLE: CHECK THE WORK

1. Start with a concrete seven-item project graph.
2. Turn it into a directory-style tree, and name the edge meaning.
3. Trace the tree three ways: preorder, postorder, and breadth-first.
4. Audit what the tree dropped; then test a decision tree's limit.

PREDICT (30 s): Which trace will visit both top-level branches before their children?

Slide legend

PREDICT = guess first;
What you should see = visible result;
Common mistake = failure mode.

Words we will use

PRINCIPLE: MODEL VS PROCEDURE

graph	A set of items and relationships between them.
vertex	One item in a graph; the dot in a picture.
directed edge	An arrow from one vertex to another; direction matters.
dependency	A relationship saying one item must be completed before another.
tree	A rooted, connected structure with one parent for each non-root item.
containment	A display relationship: one item is shown under another.
traversal	A rule for the order in which we visit vertices.
assert	A test statement that fails when a claim is false.
decision tree	A tree whose branches represent choices and outcomes.

Concrete first: a project with seven parts

PRINCIPLE: MODEL VS PROCEDURE

Imagine a small software project. Its seven parts are:

- project, plan, build
- requirements, design
- tests, docs

These are synthetic labels, not a real project.

Question

What must happen before what?

PREDICT (30 s): Which two parts can start directly after project?

A directed edge means dependency here

PRINCIPLE: EDGE MEANING

Our graph rule

An arrow `A -> B` means: complete A before B.

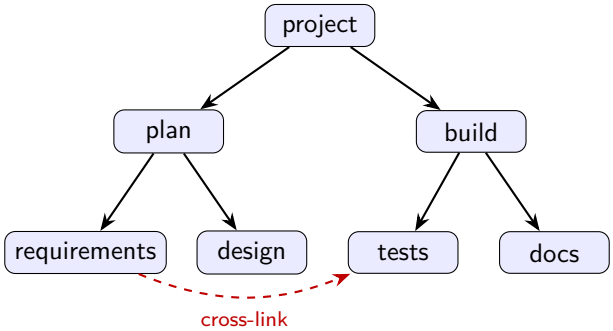
- `project -> plan`: plan follows project.
- `build -> tests`: tests follow build.
- `requirements -> tests`: tests also depend on requirements.

Common mistake: Treating every arrow as "is stored inside" changes the question.

What you should see: the arrow has a direction and a work-order meaning.

The seven-edge dependency graph

PRINCIPLE: EDGE MEANING



PREDICT (30 s): Why is `requirements -> tests` drawn across the two branches?

Read an edge before you read a trace

PRINCIPLE: EDGE MEANING

Dependency question	Containment question
Can B be completed before A?	Is B shown under A?
For `A -> B`, the answer is no.	This is a different claim.

What you should see: one drawing can answer only the question its edges were defined to answer.

PREDICT (30 s): Does the graph force tests to be drawn under requirements?

Choose a directory-style tree

PRINCIPLE: TREE STRUCTURE

Fixed display rule

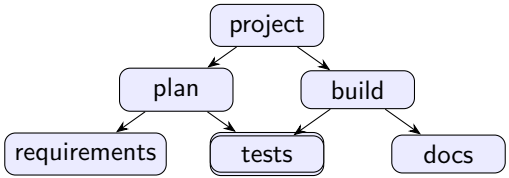
Show project with children plan and build; show requirements and design under plan; show tests and docs under build.

- Visit children left-to-right.
- Every non-root item has one parent.
- These parent-child edges mean containment: “shown under.”

PREDICT (30 s): Which graph edge will not appear as a parent-child edge?

The same items as a tree

PRINCIPLE: TREE STRUCTURE



What you should see: six parent-child edges, seven vertices, one root, and a fixed left-to-right order.

Common mistake: A parent-child line is containment here; it is not automatically a dependency.

What changed when we drew the tree?

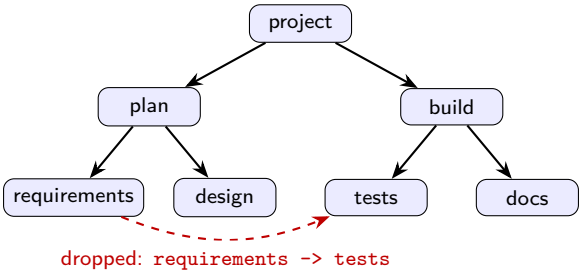
PRINCIPLE: EDGE MEANING

Question	Dependency graph	Directory-style tree
What does `A -> B` say?	Complete A before B.	Show B under A.
How many edges?	Seven listed edges.	Six parent-child edges.
Can branches cross?	Yes, `requirements -> tests`.	No, not as a tree parent relation.

What you should see: the tree is a containment view, not a complete dependency model.

The dropped cross-link is visible

PRINCIPLE: HIERARCHY LIMIT



Audit result

The tree preserves six edges and drops exactly one dependency: `requirements -> tests`.

Before code: make the audit checkable

PRINCIPLE: CHECK THE WORK

1. Write the graph edge set.
2. Write the tree parent-child edge set.
3. Subtract: graph edges minus tree edges.

```
dropped = graph_edges - tree_edges = {requirements -> tests}
```

PREDICT (30 s): What should happen if an AI-generated hierarchy claims that nothing was dropped?

Traversal is a visit-order rule

PRINCIPLE: TRAVERSAL

preorder	Visit a parent before its children.
postorder	Visit all children before their parent.
breadth-first	Visit one whole level before the next level.
left-to-right	At a node, use the child order we froze on the tree slide.

PREDICT (30 s): Which traversal reaches build before requirements?

Preorder: parent first

PRINCIPLE: TRAVERSAL

Rule

Visit the node, then recursively visit its children from left to right.

```
project, plan, requirements, design, build, tests, docs
```

What you should see: project appears before plan and build; requirements appears before design's sibling is visited.

Common mistake: Do not sort names alphabetically; use the frozen child order.

Postorder: children first

PRINCIPLE: TRAVERSAL

Rule

Finish all children before recording their parent.

`requirements, design, plan, tests, docs, build, project`

PREDICT (30 s): Which vertex must be last?

What you should see: project is last because every branch is completed before the root is recorded.

Breadth-first: level by level

PRINCIPLE: TRAVERSAL

Rule

Use a queue: visit the current level, then add the next level at the back.

```
project, plan, build, requirements, design, tests, docs
```

What you should see: project, plan, build, then requirements, design, tests, docs.

A deliberate trace error, then a fix

PRINCIPLE: CHECK THE WORK

Wrong label: “breadth-first”

```
project, plan, requirements,
design, build, tests, docs
```

Fix

```
project, plan, build, requirements,
design, tests, docs
```

This is the preorder trace wearing the wrong label.

What you should see: build belongs in the second position, before level-two nodes.

Three traces, one fixed tree

PRINCIPLE: TRAVERSAL

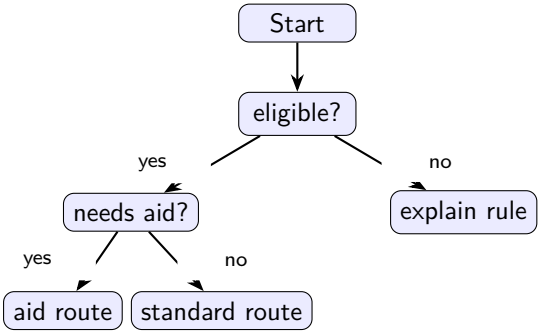
preorder	`project, plan, requirements, design, build, tests, docs`
postorder	`requirements, design, plan, tests, docs, build, project`
breadth-first	`project, plan, build, requirements, design, tests, docs`

Not used here

Inorder is a binary-tree convention; this is a general rooted tree.

Decision trees: choices on edges

PRINCIPLE: DECISION RULE



What you should see: yes leads to one next vertex and no leads to the other; both are edge labels, not extra vertices.

Trace one route: eligible and needs aid

PRINCIPLE: DECISION RULE

Input

eligible = yes; needs aid = yes

```
Start; eligible?; needs aid?; aid route
```

- Yes leads from eligible? to needs aid?.
- Yes leads from needs aid? to aid route.

What you should see: the route is a trace through nodes; yes labels each selected edge.

The three decision-tree traces

PRINCIPLE: TRAVERSAL

preorder	`Start, eligible?, needs aid?, aid route, standard route, explain rule`
postorder	`aid route, standard route, needs aid?, explain rule, eligible?, Start`
breadth-first	`Start, eligible?, needs aid?, explain rule, aid route, standard route`

Common mistake: If yes/no appears in a node list, the model has changed: labels became vertices.

The language-access omission

PRINCIPLE: HIERARCHY LIMIT

What the tree says

Eligible + needs aid leads to aid route.

What the intake note also says

The language-access review must happen before the aid route is usable.

The tree does not show that relationship.

What you should see: “aid route” is not proof that language access has been reviewed.

Bound the claim

PRINCIPLE: CHECK THE WORK

Supported claim

For this displayed tree, this child order, and this input, the trace reaches aid route.

Unsupported stronger claim

Everyone routed to aid can use the aid route immediately.

- The missing relation may need another node: language-access review.
- Or it must remain an explicit limitation of the model.

Prompt 01a: model the graph

PRINCIPLE: MODEL VS PROCEDURE

PROMPT FOR CLAUDE

prompts/01_project_graph.md

```
Predict which seven vertices and seven directed edges you expect before you run anything.
Use the synthetic project graph with vertices project, plan, build, requirements, design,
tests, and docs. In this lesson, a directed edge A -> B means ‘‘A must be completed
before B.’’ Write a small Python module that stores exactly these edges:
project -> plan, project -> build, plan -> requirements, plan -> design,
build -> tests, build -> docs, and requirements -> tests. Print the vertices and the
sorted edge list. Run the module. Explain how your output matches the prediction.
Do not use real project, student, employee, or host data.
```

PREDICT (30 s): Before running: which exact seven edges should the test require?

What you should see: the model is written down before the code is trusted.

Prompt 01b: test the graph

PRINCIPLE: MODEL VS PROCEDURE

PROMPT FOR CLAUDE

prompts/01_project_graph.md

```
Predict what an exact edge-set test should catch. Add pytest tests for the seven
synthetic vertices and the exact seven-edge set listed in prompt-a. Do not edit these
tests later, even if a later prompt produces a surprising result. Run pytest and show
the short result. Explain what an exact set check tells us that a picture alone may not.
```

What you should see: the exact-set test checks the graph without allowing later code to change it.

Prompt 02a: draw the directory tree

PRINCIPLE: TREE STRUCTURE

PROMPT FOR CLAUDE

prompts/02_directory_tree.md

```
Predict which relationships will be shown by this directory-style tree:
project has children plan and build; plan has children requirements and design;
build has children tests and docs. Convert the seven-vertex synthetic graph into
that exact tree, with children visited left-to-right. State plainly that a tree edge
means containment (‘‘is shown under’’), while the original graph edge means dependency
(‘‘must be completed before’’). Print each parent and its children. Run it, then explain
which graph relation is not shown by the tree.
```

PREDICT (30 s): Before running: which edge changes meaning when it becomes a parent-child line?

What you should see: the picture is a containment view, so its edges need a new definition.

Prompt 02b: test the tree

PRINCIPLE: TREE STRUCTURE

PROMPT FOR CLAUDE

prompts/02_directory_tree.md

```
Predict the checks a valid tree should pass. Add pytest tests that check the root is
project, every non-root vertex has one parent, all seven vertices appear exactly once,
and the tree is acyclic. Check the fixed child order project=(plan, build),
plan=(requirements, design), and build=(tests, docs). Run pytest and explain each
check using only the synthetic tree.
```

Common mistake: A neat directory picture is not automatically a complete dependency graph.

Prompt 03a: trace the tree

PRINCIPLE: TRAVERSAL

PROMPT FOR CLAUDE

prompts/03_traversals.md

```
Predict the visit order for preorder (parent before children), postorder (children
before parent), and breadth-first (level by level) on the synthetic project tree.
Implement those three traversals without changing the child order. Run them from
project and print the three traces. Explain why inorder is not used: inorder belongs
to binary-tree conventions, while this is a general rooted tree.
```

PREDICT (30 s): Before running: which list should put build before requirements?

What you should see: the three traversal rules answer three different visit-order questions.

Prompt 03b: test the traces

PRINCIPLE: TRAVERSAL

PROMPT FOR CLAUDE

prompts/03_traversals.md

```
Before running tests, paste YOUR predictions into these three lines:
preorder: [your predicted list]
postorder: [your predicted list]
breadth-first: [your predicted list]
Add pytest assertions for the exact verified traces. Use the reference module's
fixed child-order constants (such as TREE_CHILDREN) and any trace constants it
provides where possible; if a verified list must be recorded, take it from the
reference run, not from these prediction lines. Keep the pasted predictions as
notes for comparison, not as the test oracle. Run pytest and explain why a
depth-first implementation cannot produce the breadth-first trace.
```

What you should see: the breadth-first list is the level-by-level list generated from the fixed order.

Prompt 04a: trace a decision tree

PRINCIPLE: DECISION RULE

PROMPT FOR CLAUDE

prompts/04_decision_tree_aid_language.md

```
Predict the route for a synthetic person who is eligible and needs aid. Model this
decision tree: Start has child eligible?; from eligible?, yes leads to needs aid?
and no leads to explain rule; from needs aid?, yes leads to aid route and no leads
to standard route. Store yes and no as edge labels, not as vertices. Add a note
linking aid route and language-access review, because that relation is not in the
displayed tree. State plainly that the language-access review must happen before
the aid route is usable. Run a trace for eligible=yes and needs aid=yes. Explain
which step the tree shows and which language-access condition it does not show.
Use no real intake data.
```

PREDICT (30 s): Before running: should yes or no appear in the node trace?

What you should see: the route records nodes; yes and no label which edge leads to the next node.

Prompt 04b: test the decision tree

PRINCIPLE: DECISION RULE

PROMPT FOR CLAUDE

prompts/04_decision_tree_aid_language.md

```
Predict the route and the missing relationship before testing. Add pytest tests for
the eligible-plus-aid trace, the exact preorder and breadth-first traces of the
decision tree, and the fact that yes and no are edge labels rather than nodes.
Postorder is optional for this decision-tree check. Use the reference module's
decision-tree constants where possible, and do not put yes or no into the node
lists. Run pytest. Explain why ''aid route'' is not enough to prove that the review
has happened before the route is usable.
```

What you should see: the trace ends at aid route; the review-before-use condition is a separate note.

Prompt 05a: audit the cross-link

PRINCIPLE: CHECK THE WORK

PROMPT FOR CLAUDE

prompts/05_audit_cross_link.md

```
Predict whether the directory-style tree contains every dependency from the
synthetic graph. Before running, paste your predicted dropped edge set here:
dropped: [your predicted set]. Compare the exact graph edge set with the tree
parent-child edge set. Run the audit and print the actual dropped set. Compare it
with the prediction and explain any difference. Explain the AI failure mode: a
clean hierarchy can erase a relevant relation, because a dependency edge can
cross from one branch to another even when the tree looks tidy. Do not use a real
project or any personal data.
```

PREDICT (30 s): Before running: what dropped edge set do you predict?

What you should see: set subtraction compares relationships, not just the names of nodes.

Prompt 05b: add the audit test

PRINCIPLE: CHECK THE WORK

PROMPT FOR CLAUDE

prompts/05_audit_cross_link.md

```
Predict whether the regression test will name a missing edge. Add a pytest test
that asserts the audit result from the reference module's GRAPH_EDGES and
TREE_EDGES constants, rather than using your prediction as the oracle. Also add a
clearly labeled strict expected-failure test for the intentionally false claim
that a hierarchy preserves every graph dependency; use
pytest.mark.xfail(strict=True, reason=...). Run pytest and explain why the
expected failure is evidence about the claim, not a broken test suite. Do not
replace the expected-failure marker with a passing test.
```

What you should see: the audit names the actual dropped relation; the strict expected failure stays visible.

Prompt 06a: check a search claim

PRINCIPLE: HIERARCHY LIMIT

PROMPT FOR CLAUDE

prompts/06_search_claim_check.md

```
Predict which searches should succeed before running them. Write a search helper for
the seven synthetic tree nodes and test searches for project, requirements, tests,
and docs. Then test a missing-node search for language-access review. Run the helper
and print each result. Explain that finding a node is not the same as proving every
dependency involving that node is present.
```

PREDICT (30 s): Before running: can finding every known node prove every dependency is present?

What you should see: a node search asks whether an item is present; an edge audit asks whether a relationship is present.

Prompt 06b: test the stronger claim

PRINCIPLE: HIERARCHY LIMIT

PROMPT FOR CLAUDE

prompts/06_search_claim_check.md

```
Predict that all seven known nodes will be found and the missing node will not. Add
pytest assertions for those cases. Then assert that the tree cannot support the
stronger claim ‘‘all dependencies are preserved,’’ because the exact audit reports
requirements -> tests as absent. Run pytest and explain the difference between a
successful node search and a complete relationship audit.
```

Common mistake: A successful node search is not a relationship audit.

Prompt 07a: explain the limit

PRINCIPLE: CHECK THE WORK

PROMPT FOR CLAUDE

prompts/07_explain_limit.md

```
Predict the evidence you will print before running the complete suite. Run the full
pytest suite and print the project preorder, postorder, and breadth-first traces, the
decision-tree traces, and the dropped dependency set. Then write YOUR OWN
one-sentence conclusion and check it against the evidence: say what the tree
supports and what limitation follows from any dropped relationship. Do not copy a
conclusion from this prompt. Explain which result is a direct check and which
statement is a limitation of the model. Use synthetic data only, and do not edit
tests while explaining the result.
```

PREDICT (30 s): Before running: what evidence will support the one-sentence conclusion?

What you should see: your conclusion matches the traces, the dropped dependency, and the limits of those checks.

Audit table: what survived?

PRINCIPLE: CHECK THE WORK

Source relationship	Tree display	Result
`project -> plan`	project contains plan	preserved in display
`project -> build`	project contains build	preserved in display
`plan -> requirements`	plan contains requirements	preserved in display
`plan -> design`	plan contains design	preserved in display
`build -> tests`	build contains tests	preserved in display
`build -> docs`	build contains docs	preserved in display
`requirements -> tests`	different branches	dropped

What you should see: the exact audit output is {`requirements -> tests`}.

One bounded conclusion

PRINCIPLE: HIERARCHY LIMIT

Conclusion

The fixed tree is a useful containment view with reproducible traversal orders, but it does not preserve the `requirements -> tests` dependency.

- Direct checks: exact node sets, exact traces, and exact dropped-edge set.
- Limitation: the tree cannot answer a relationship question it does not represent.

Decision Gate preview: part 1

PRINCIPLE: CHECK THE WORK

```
# Week 13 Decision Gate --- A tree that earns its conclusion

Submit one readable individual artifact. You may draw it clearly on paper and
scan it, or use a local text/diagram format. No particular software is
required.

Choose a small supplied or personally invented **non-sensitive** search,
classification, or decision situation. Do not use private student, employee,
health, or financial data. Build a rooted tree with at least five labeled
nodes and make one claim it can support.

Use these headings:

1. **Sources** --- identify the supplied scenario or explain the bounded
   invented scenario.
```

Decision Gate preview: part 2

PRINCIPLE: CHECK THE WORK

2. **Rules / Assumptions** --- define what an edge and leaf mean; state the decision/search rule and relevant assumptions.
3. **Work** --- show the labeled tree and two traversal or search traces, including one edge case.
4. **Check Your Answer** --- test the claim using a second trace, a counterexample attempt, or comparison to the original non-tree relationships. Name one relation the tree leaves out.
5. **One-Sentence Summary** --- state the conclusion and its limit.

The artifact is the Week 13 ordinary Decision Gate in the 30% Decision Gate category. It is not an Odyssey Checkpoint and does not add a second technical submission. Use ‘templates/weekly-rubric.md‘ as the evaluation shape.

The exact submission location and due time are published through the course site; do not infer them from this source package.

Thursday bridge: audit search claims

PRINCIPLE: DECISION RULE

Tuesday leaves us with

A tree trace and a named missing edge.

Thursday asks us to

Pair-reason about a search claim, compare it with the original relationships, and report the limit.

What you should see: the Thursday report maps back to the same Sources, Rules, Work, Check, Summary method.

Close: model before procedure

PRINCIPLE: MODEL VS PROCEDURE

Say it in one sentence

A tree can make a structure easy to see while hiding a relationship it was not built to represent.

- Define the edge.
- Freeze the order.
- Predict, run, explain.
- Audit the claim against the original relationships.

Index 1 of 5: principle, slide, prompt

Slide	Title	Principle	Prompt
2	Week 12 continuity: the model comes first	MODEL VS PROCEDURE	
3	Today: model, trace, audit	CHECK THE WORK	
4	Words we will use	MODEL VS PROCEDURE	
5	Concrete first: a project with seven parts	MODEL VS PROCEDURE	
6	A directed edge means dependency here	EDGE MEANING	
7	The seven-edge dependency graph	EDGE MEANING	
8	Read an edge before you read a trace	EDGE MEANING	
9	Choose a directory-style tree	TREE STRUCTURE	
10	The same items as a tree	TREE STRUCTURE	

Index 2 of 5: principle, slide, prompt

Slide	Title	Principle	Prompt
11	What changed when we drew the tree?	EDGE MEANING	
12	The dropped cross-link is visible	HIERARCHY LIMIT	
13	Before code: make the audit checkable	CHECK THE WORK	
14	Traversal is a visit-order rule	TRAVERSAL	
15	Preorder: parent first	TRAVERSAL	
16	Postorder: children first	TRAVERSAL	
17	Breadth-first: level by level	TRAVERSAL	
18	A deliberate trace error, then a fix	CHECK THE WORK	
19	Three traces, one fixed tree	TRAVERSAL	

Index 3 of 5: principle, slide, prompt

Slide	Title	Principle	Prompt
20	Decision trees: choices on edges	DECISION RULE	
21	Trace one route: eligible and needs aid	DECISION RULE	
22	The three decision-tree traces	TRAVERSAL	
23	The language-access omission	HIERARCHY LIMIT	
24	Bound the claim	CHECK THE WORK	
25	Prompt 01a: model the graph	MODEL VS PROCEDURE	01a
26	Prompt 01b: test the graph	MODEL VS PROCEDURE	01b
27	Prompt 02a: draw the directory tree	TREE STRUCTURE	02a
28	Prompt 02b: test the tree	TREE STRUCTURE	02b

Index 4 of 5: principle, slide, prompt

Slide	Title	Principle	Prompt
29	Prompt 03a: trace the tree	TRAVERSAL	03a
30	Prompt 03b: test the traces	TRAVERSAL	03b
31	Prompt 04a: trace a decision tree	DECISION RULE	04a
32	Prompt 04b: test the decision tree	DECISION RULE	04b
33	Prompt 05a: audit the cross-link	CHECK THE WORK	05a
34	Prompt 05b: add the audit test	CHECK THE WORK	05b
35	Prompt 06a: check a search claim	HIERARCHY LIMIT	06a
36	Prompt 06b: test the stronger claim	HIERARCHY LIMIT	06b
37	Prompt 07a: explain the limit	CHECK THE WORK	07a

Index 5 of 5: principle, slide, prompt

Slide	Title	Principle	Prompt
38	Audit table: what survived?	CHECK THE WORK	
39	One bounded conclusion	HIERARCHY LIMIT	
40	Decision Gate preview: part 1	CHECK THE WORK	
41	Decision Gate preview: part 2	CHECK THE WORK	
42	Thursday bridge: audit search claims	DECISION RULE	
43	Close: model before procedure	MODEL VS PROCEDURE	